



CS 498: Pipeline Parallelism

Fan Lai

The Grainger College of Engineering

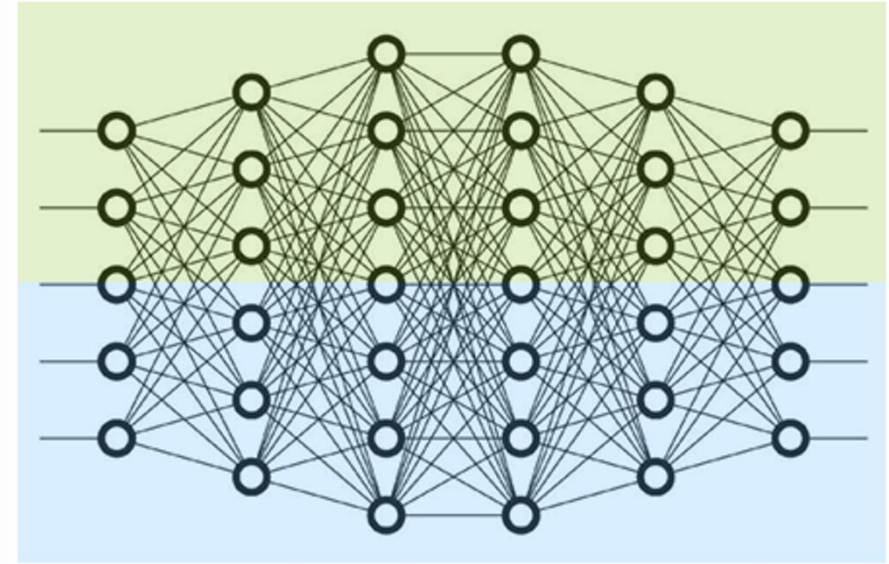
Materials with thanks to Minjia Zhang, Arvind Krishnamurthy,
Tianqi Chen and many colleagues

Recall: Tensor Parallelism is Communication-heavy



Intra-layer (Tensor) parallelism

- Split individual layers across multiple devices
- Both devices compute different parts of layer 0, 1, 2, 3, 4, 5

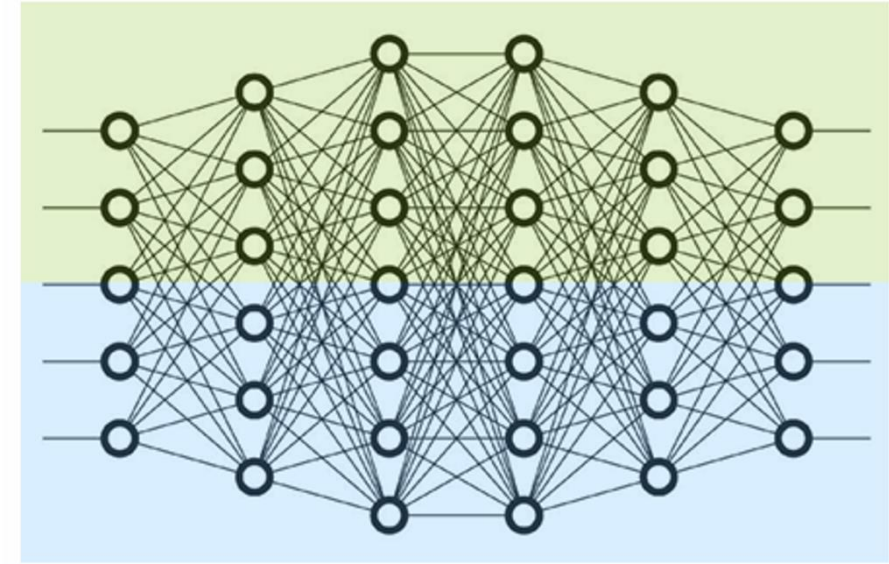


Recall: Tensor Parallelism is Communication-heavy



Intra-layer (Tensor) parallelism

- Split individual layers across multiple devices
- Both devices compute different parts of layer 0, 1, 2, 3, 4, 5



Row-wise sharding (of A):

$$\begin{pmatrix} X_0 & X_1 \\ X_2 & X_3 \end{pmatrix} \begin{pmatrix} A_0 & A_1 \\ A_2 & A_3 \end{pmatrix} = \begin{pmatrix} X_0 \\ X_2 \end{pmatrix} (A_0 \ A_1) + \begin{pmatrix} X_1 \\ X_3 \end{pmatrix} (A_2 \ A_3)$$
$$= \underbrace{X_A A_A + X_B A_B}_{\text{Summation}}$$

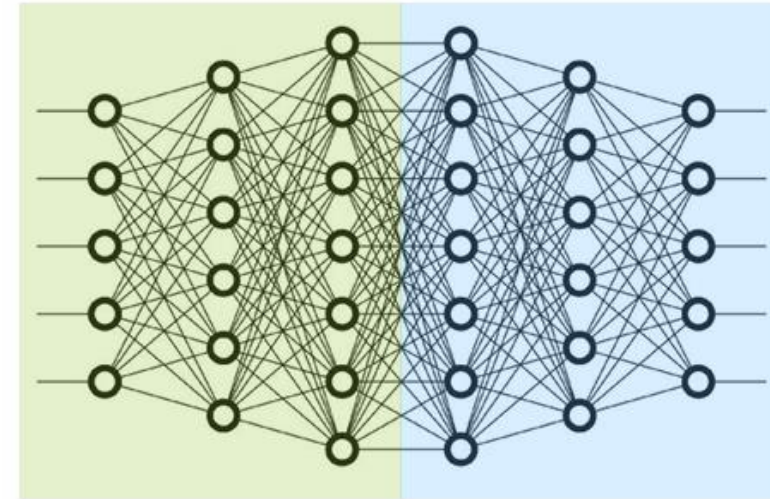
Column-wise sharding (of A):

$$\begin{pmatrix} X_0 & X_1 \\ X_2 & X_3 \end{pmatrix} \begin{pmatrix} A_0 & A_1 \\ A_2 & A_3 \end{pmatrix} = \left(\begin{pmatrix} X_0 & X_1 \\ X_2 & X_3 \end{pmatrix} \begin{pmatrix} A_0 \\ A_2 \end{pmatrix} \quad \begin{pmatrix} X_0 & X_1 \\ X_2 & X_3 \end{pmatrix} \begin{pmatrix} A_1 \\ A_3 \end{pmatrix} \right)$$
$$= \underbrace{(X A_A \quad X A_B)}_{\text{Concatenation}}$$

Pipeline Parallelism

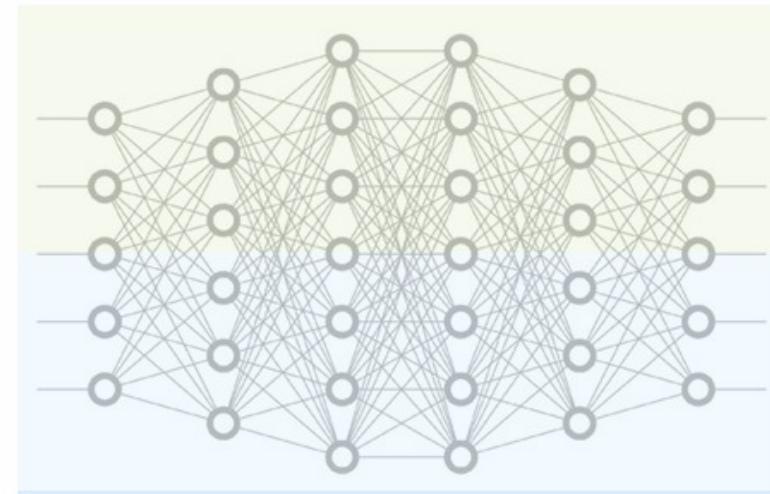
Inter-layer (Pipeline) parallelism

- Split sets of layers across multiple devices
- Layer 0, 1, 2 and layer 3, 4, 5 are on different devices

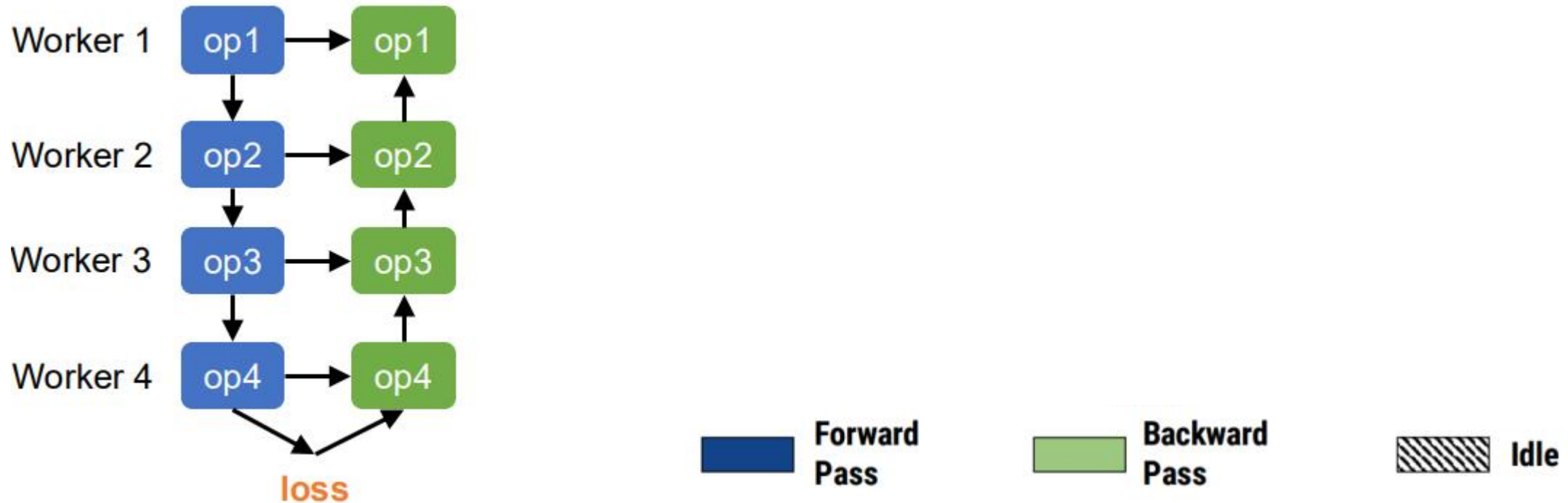


Intra-layer (Tensor) parallelism

- Split individual layers across multiple devices
- Both devices compute different parts of layer 0, 1, 2, 3, 4, 5



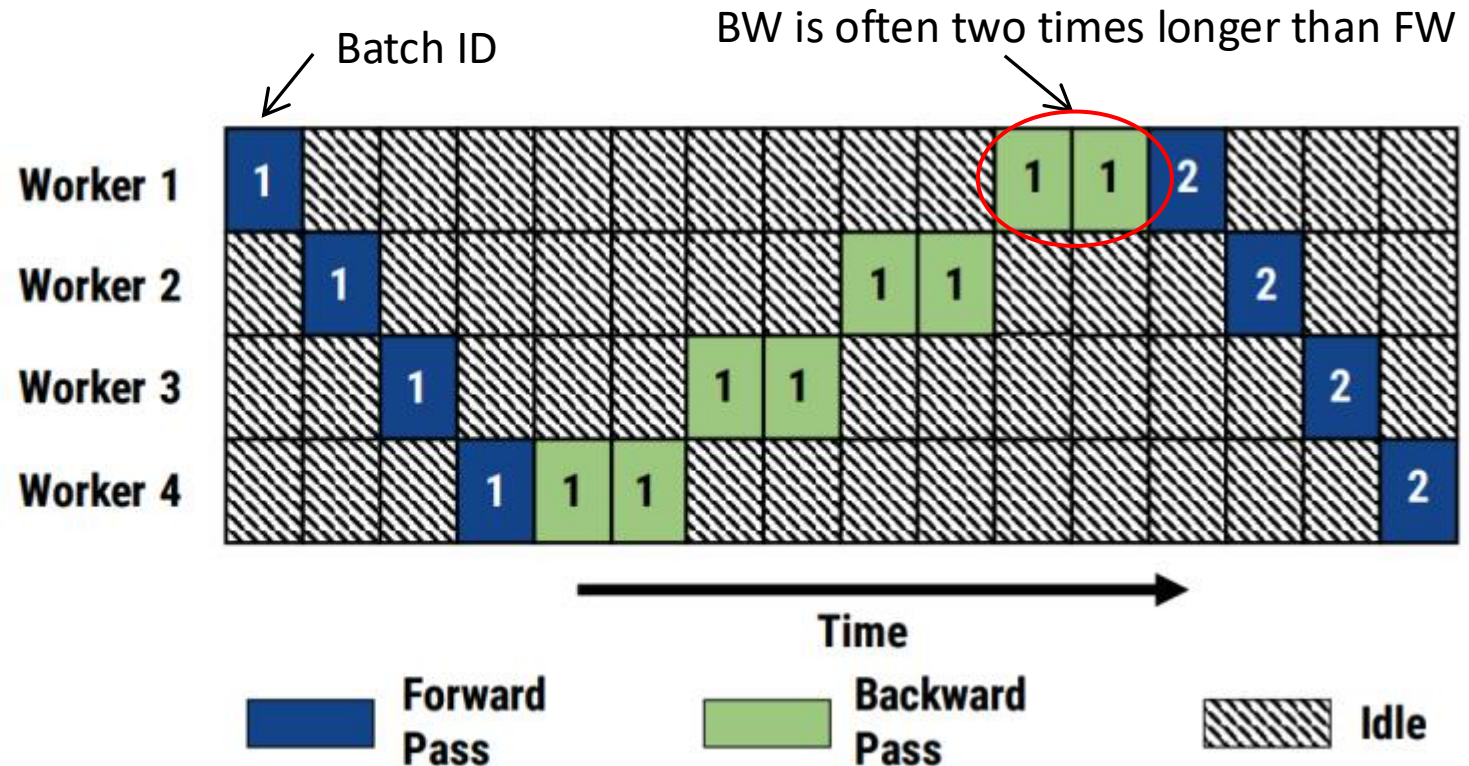
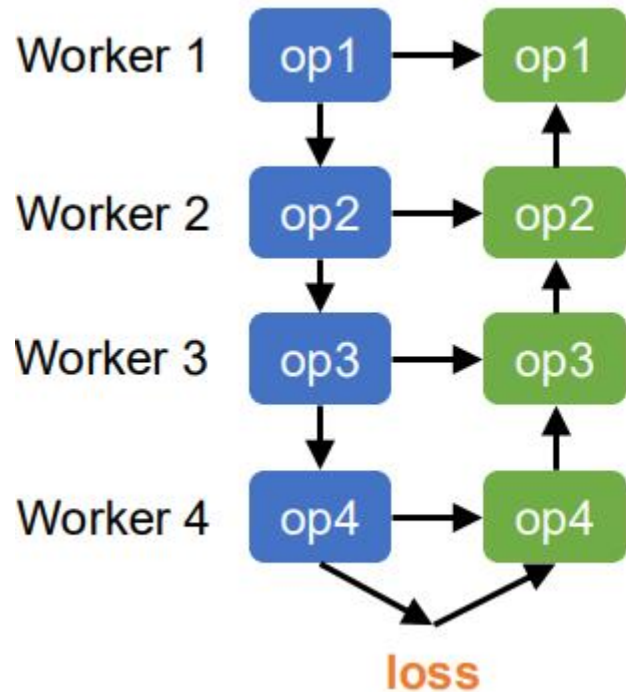
Simple Inter-Layer Parallelism



DNN training involves a **bi-directional** execution

- The forward pass for a minibatch starts at the input layer
- The backward pass ends at the input layer

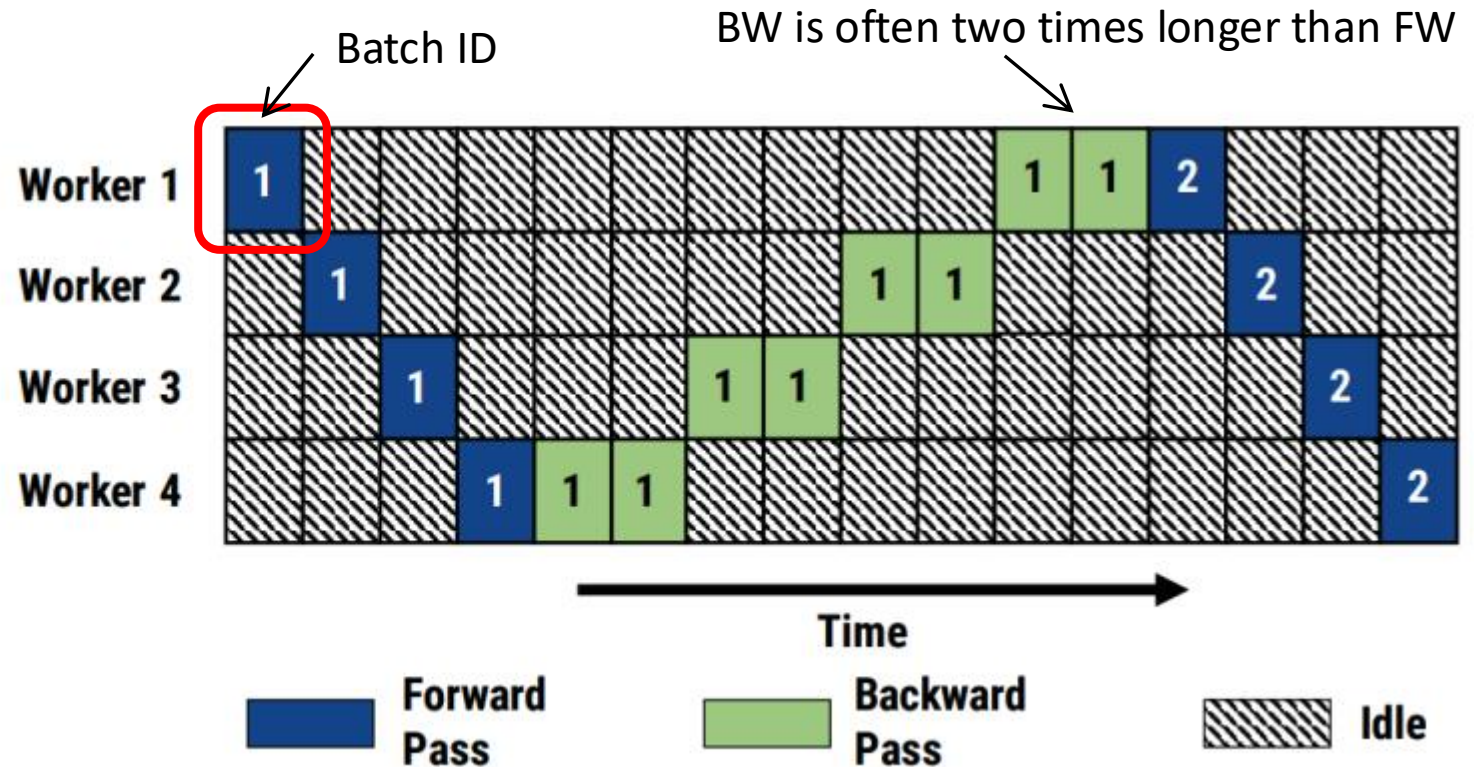
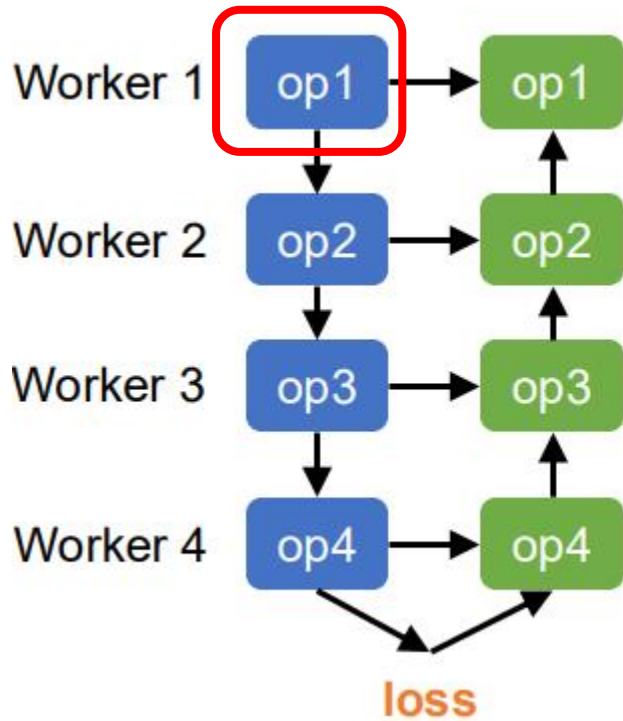
Simple Inter-Layer Parallelism



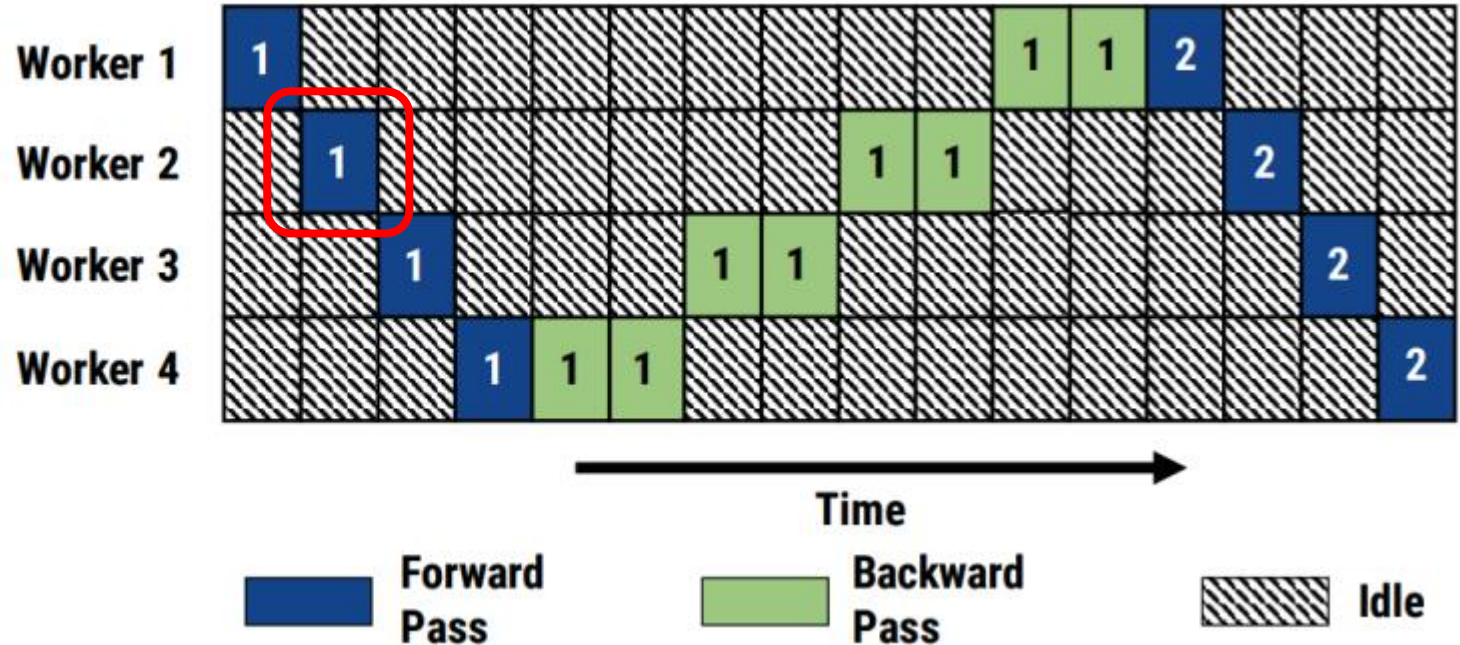
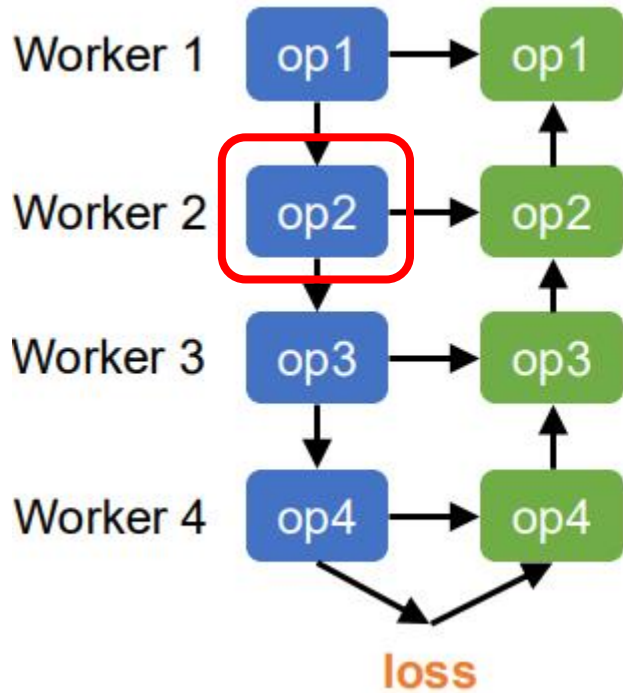
DNN training involves a **bi-directional** execution

- The forward pass for a minibatch starts at the input layer
- The backward pass ends at the input layer

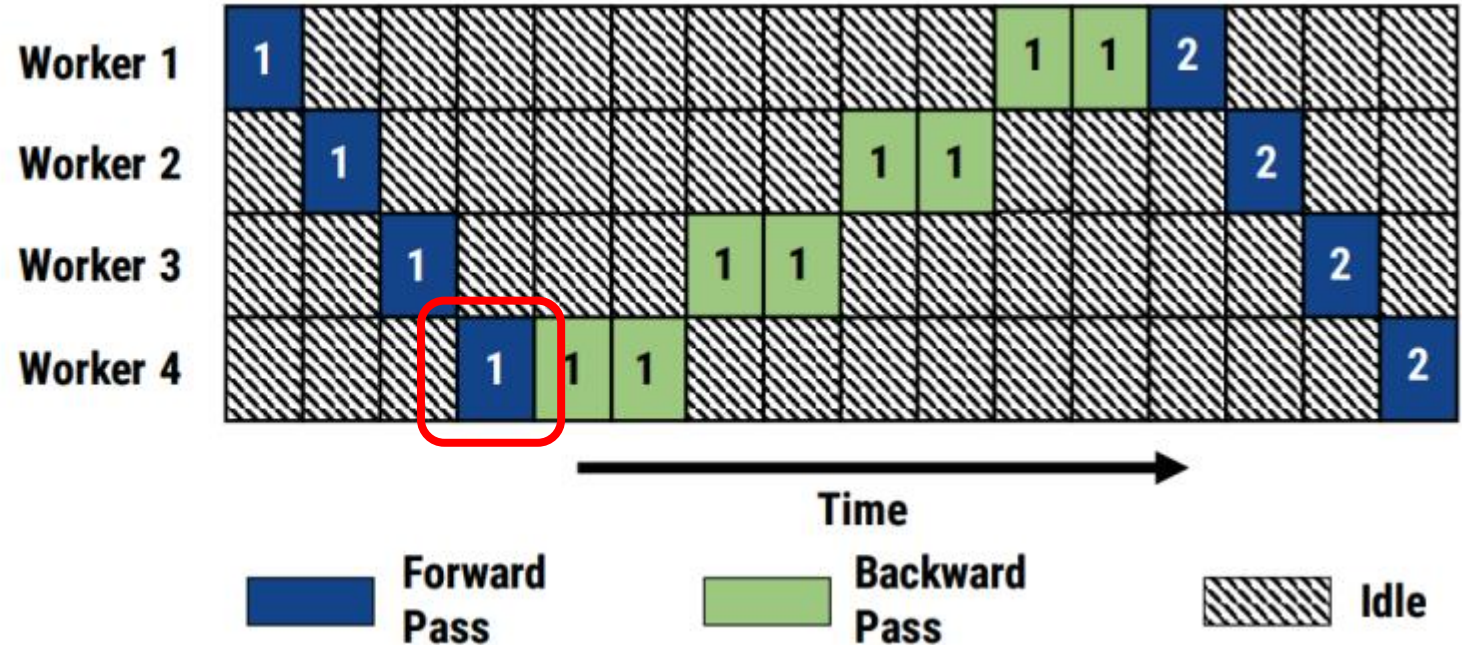
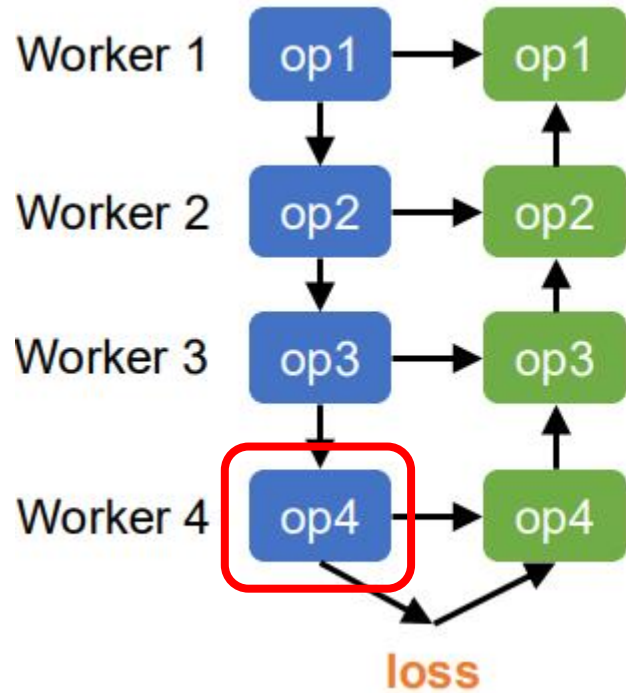
Simple Inter-Layer Parallelism



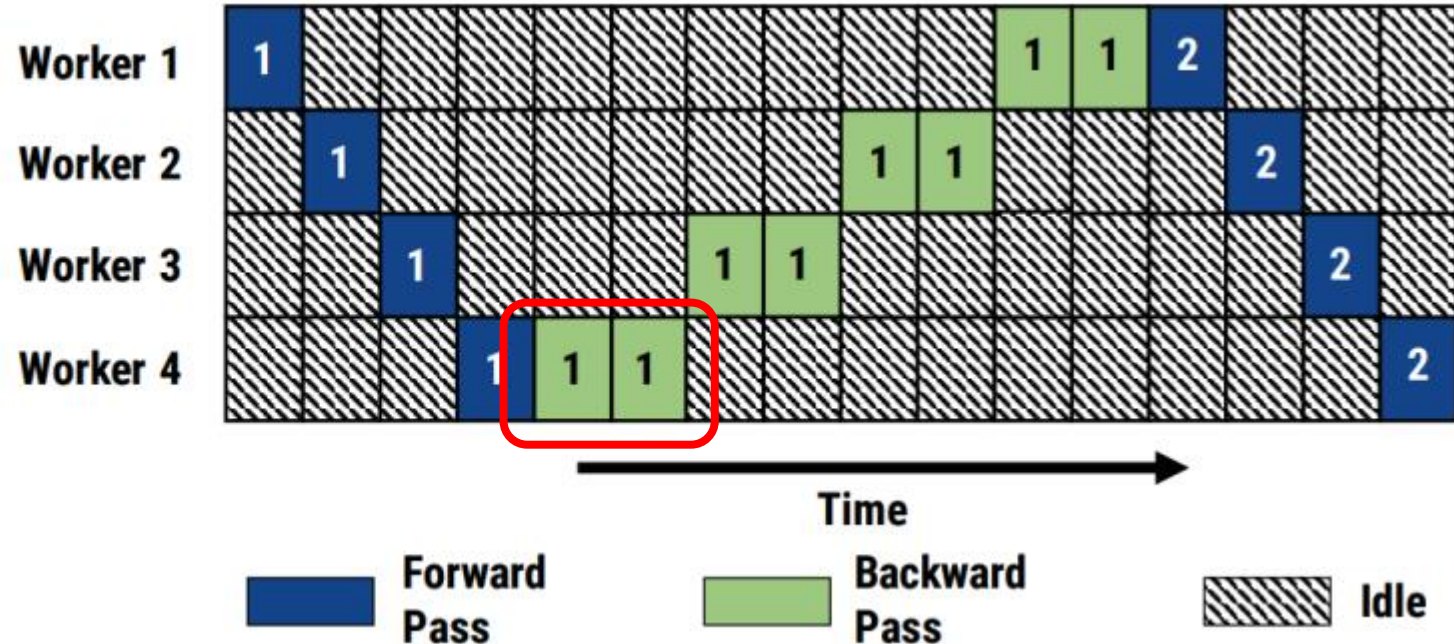
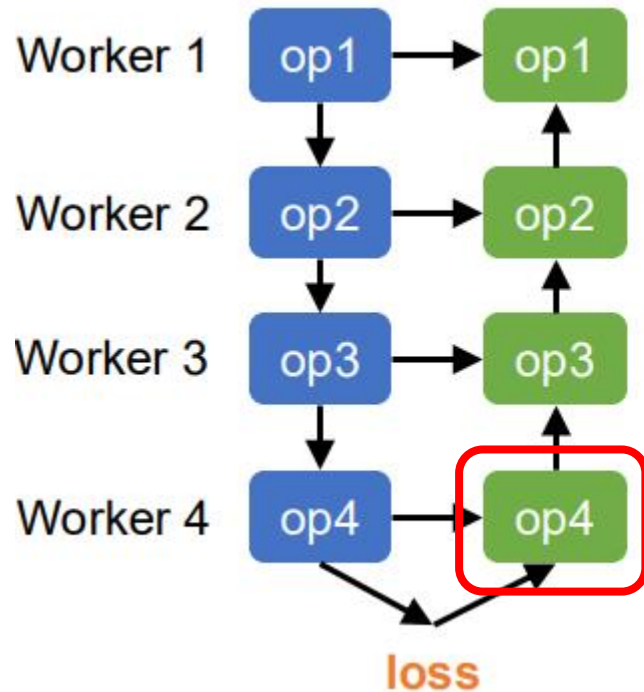
Simple Inter-Layer Parallelism



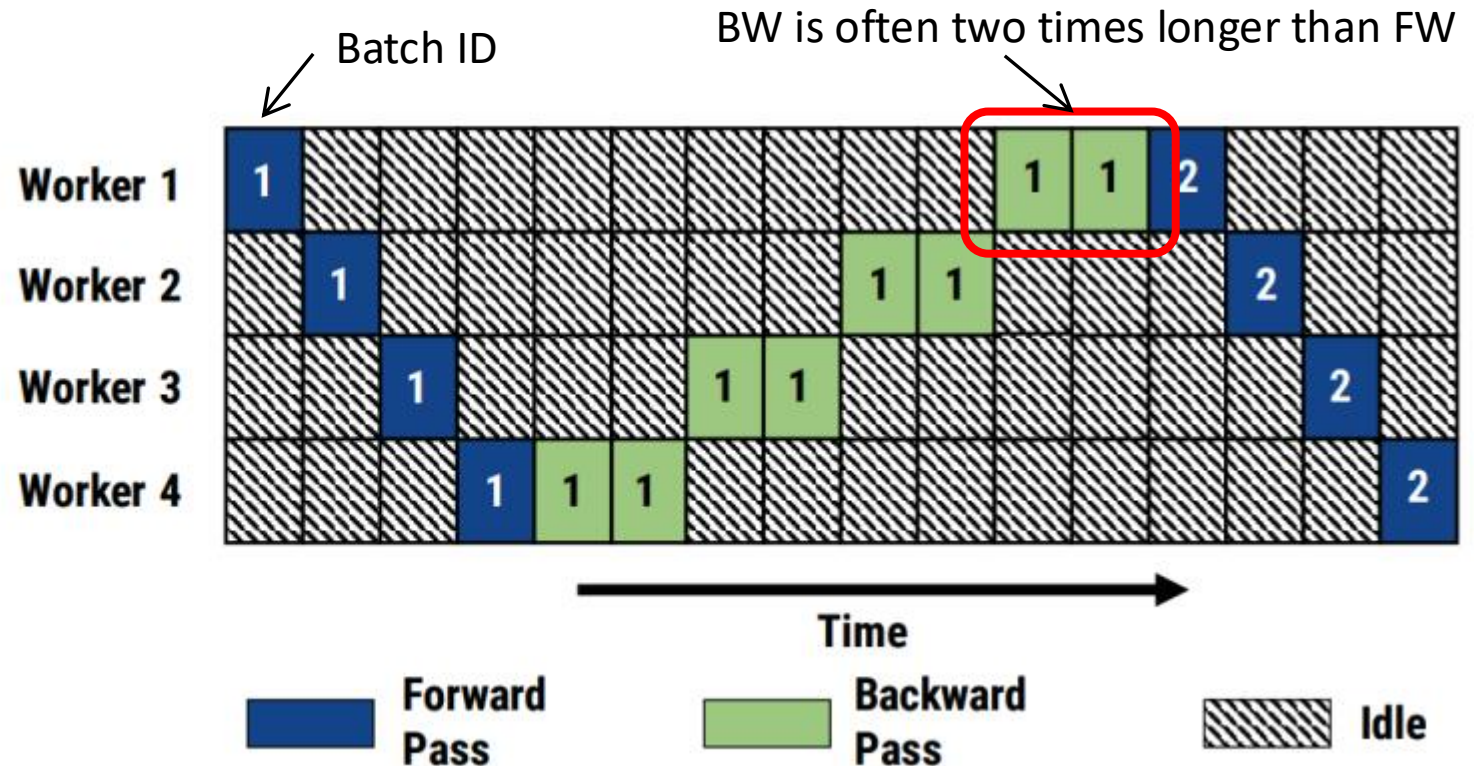
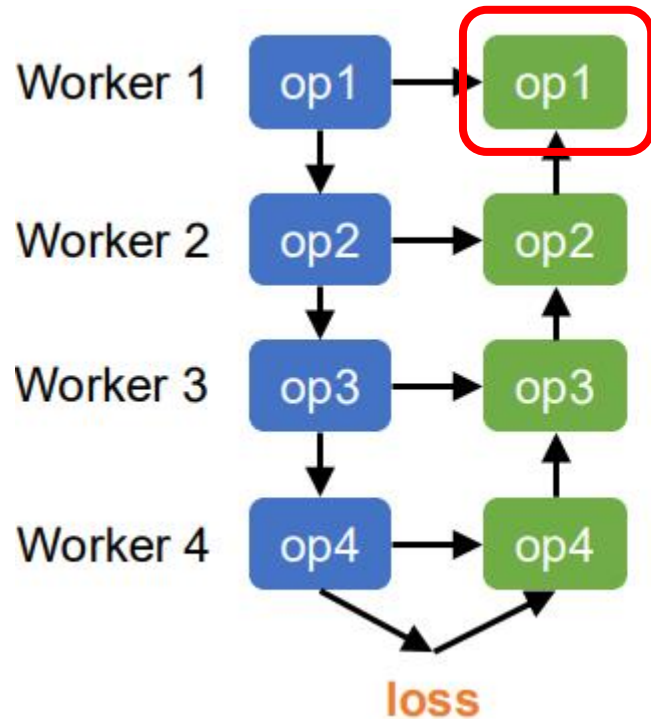
Simple Inter-Layer Parallelism



Simple Inter-Layer Parallelism

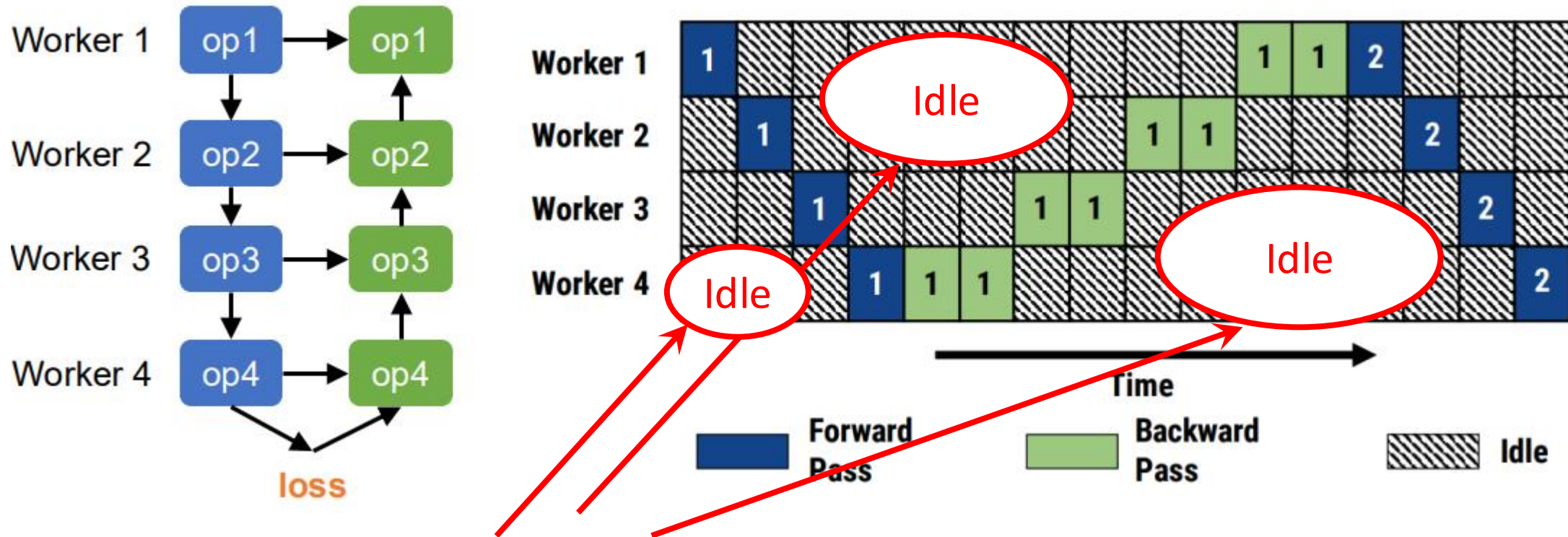


Simple Inter-Layer Parallelism



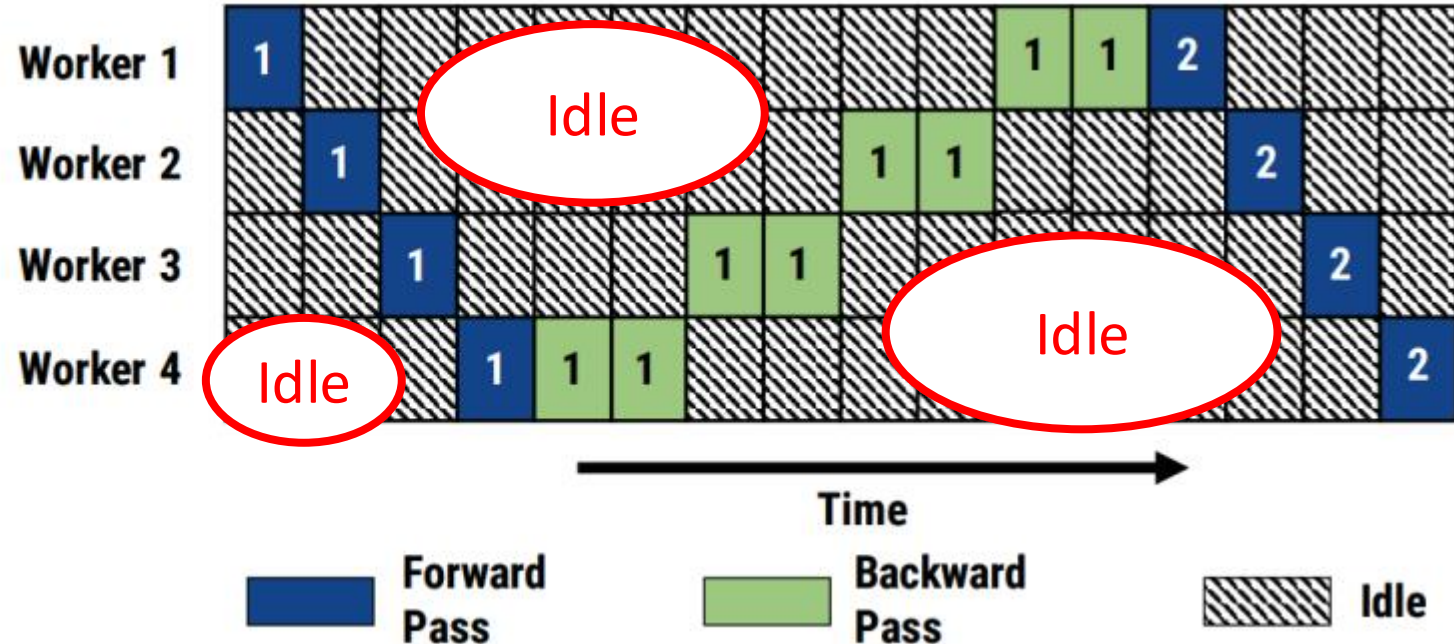
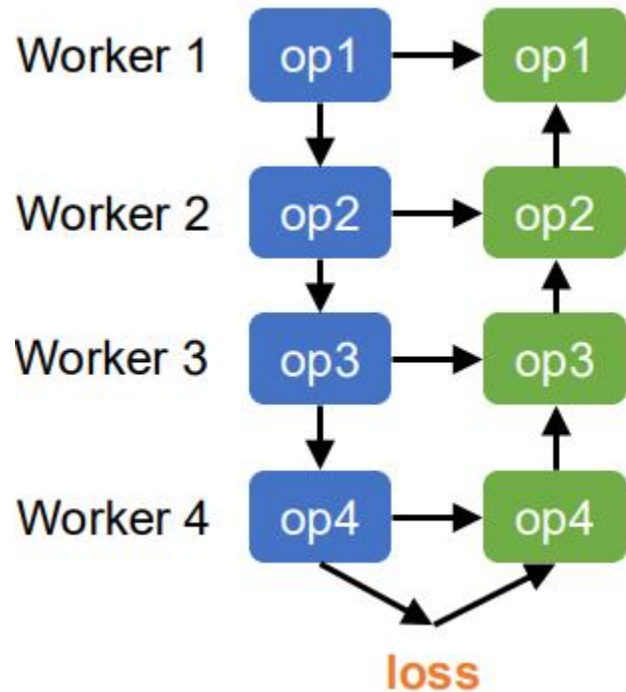
Question: What is the limitation of this execution?

Issues with Simple Inter-Layer Parallelism



- **Under-utilization** of compute resources
- Only one device is computing at a time and others are idling

Issues with Simple Inter-Layer Parallelism

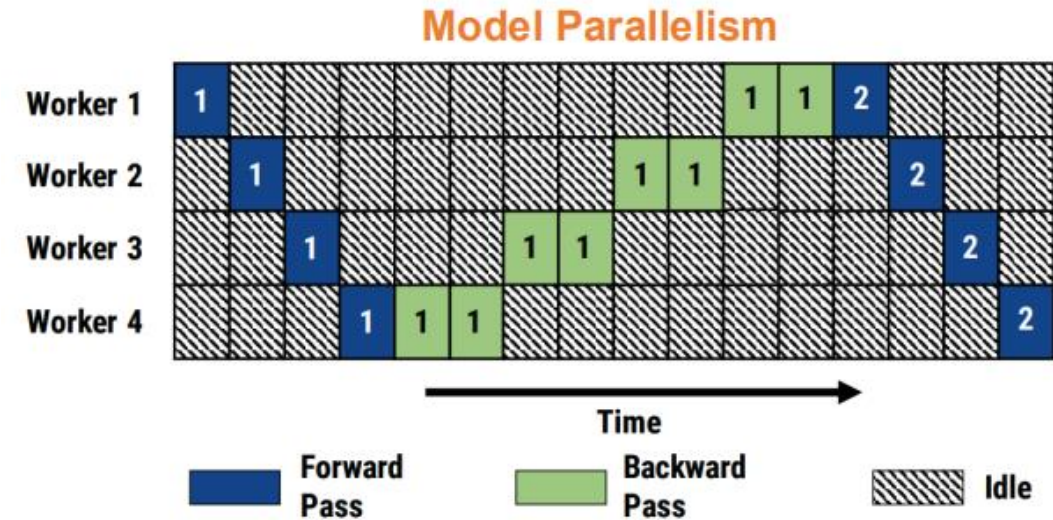


Question: How to improve the utilization and let multiple workers work simultaneously?

Pipeline Model Parallelism



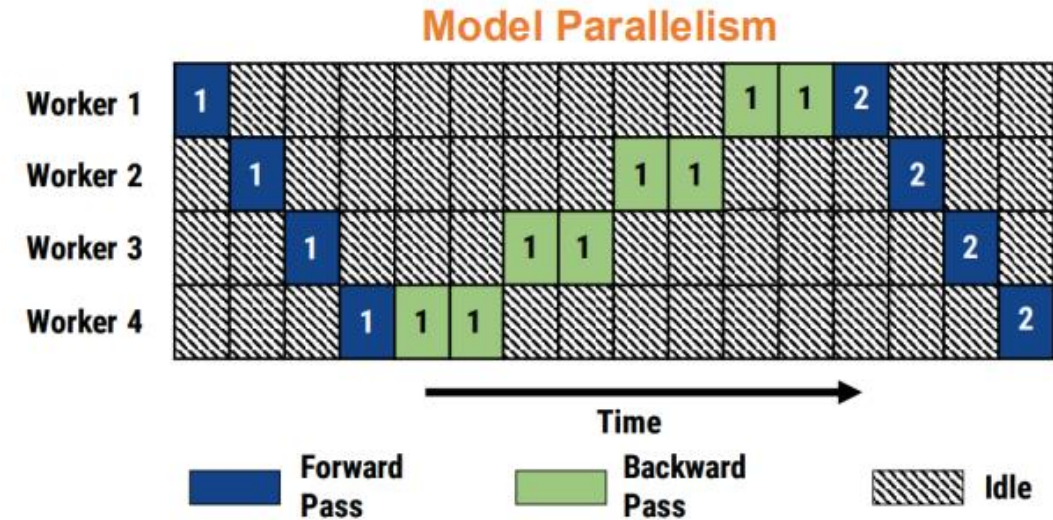
- **Mini-batch**: the number of samples processed in each iteration
- Divide a mini-batch into multiple smaller **micro-batches**



Pipeline Model Parallelism



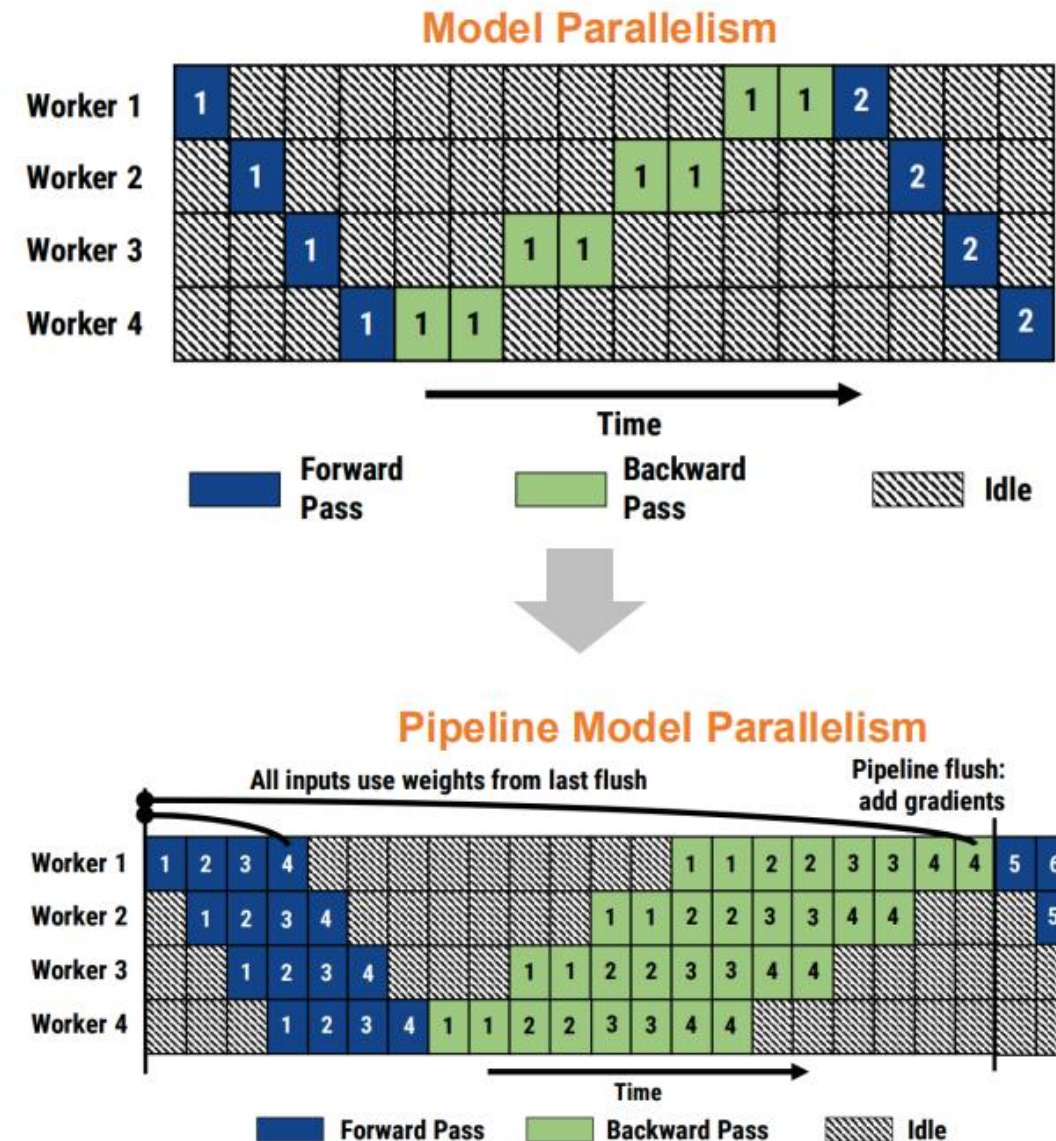
- **Mini-batch**: the number of samples processed in each iteration
- Divide a mini-batch into multiple smaller **micro-batches** (e.g., $128=4*32$)



Pipeline Model Parallelism



- **Mini-batch**: the number of samples processed in each iteration
- Divide a mini-batch into multiple smaller **micro-batches** (e.g., $128=4*32$)
- **Pipeline execution**:
 - Micro-batches flow through the pipeline from one stage to the next.
 - As soon as a stage completes its work, it passes the micro-batch to the next stage and starts working on the next micro-batch



Pipeline Model Parallelism: Device Utilization



m = #microbatches (8)

p = pipeline stages (4)

t_f = time of forward

t_b = time of backward



Pipeline Model Parallelism: Device Utilization



m = #microbatches (8)

p = pipeline stages (4)

t_f = time of forward

t_b = time of backward



Pipeline Model Parallelism: Device Utilization

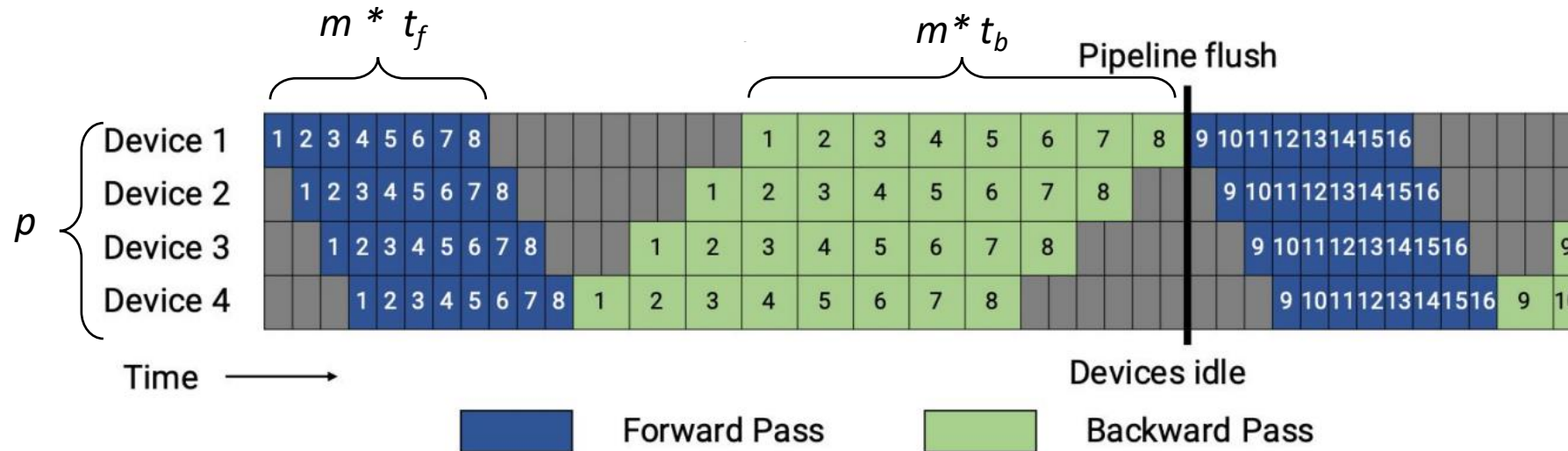


m = #microbatches (8)

p = pipeline stages (4)

t_f = time of forward

t_b = time of backward



Pipeline Model Parallelism: Device Utilization



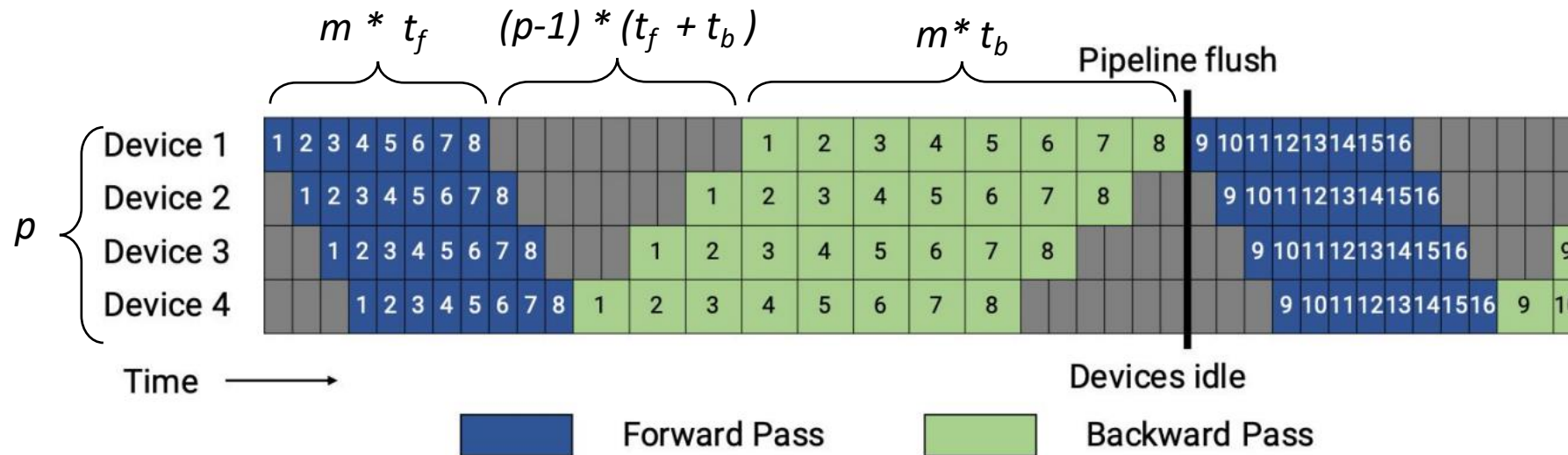
m = #microbatches (8)

p = pipeline stages (4)

t_f = time of forward

t_b = time of backward

Each device needs to wait for $(p - 1)$ devices to finish their forward and backward passes



Pipeline Model Parallelism: Device Utilization

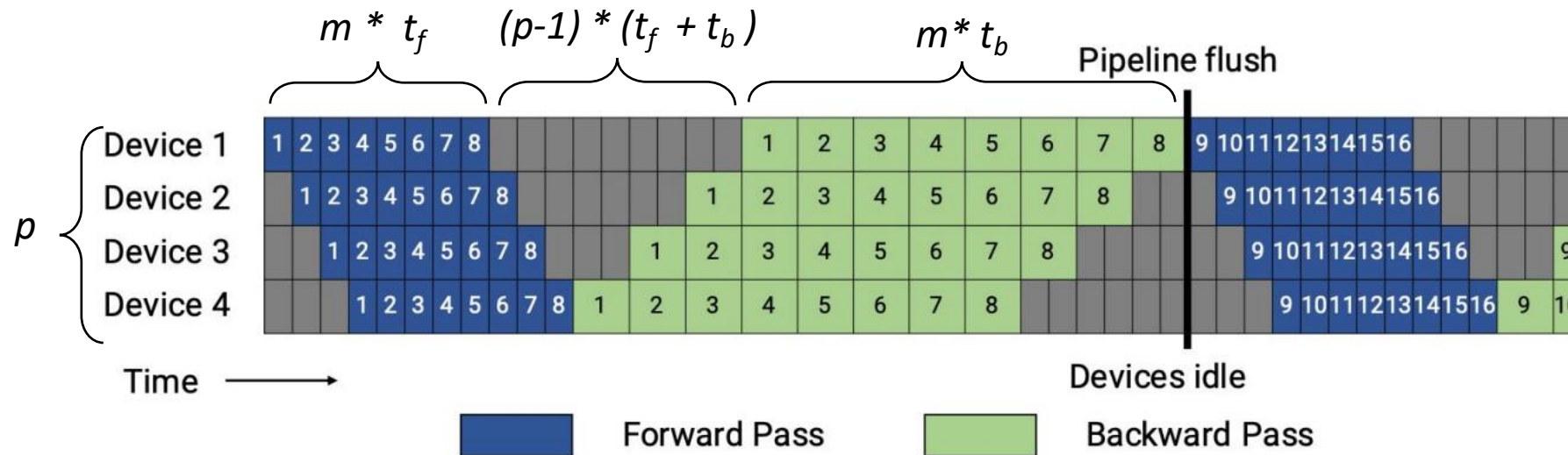


m = #microbatches (8)

p = pipeline stages (4)

t_f = time of forward

t_b = time of backward



$$\text{BubbleFraction} = \frac{(p-1) * (t_f + t_b)}{\boxed{m * t_f + m * t_b}} = \frac{p-1}{m}$$

Amount of real execution time

Pipeline Model Parallelism: Device Utilization

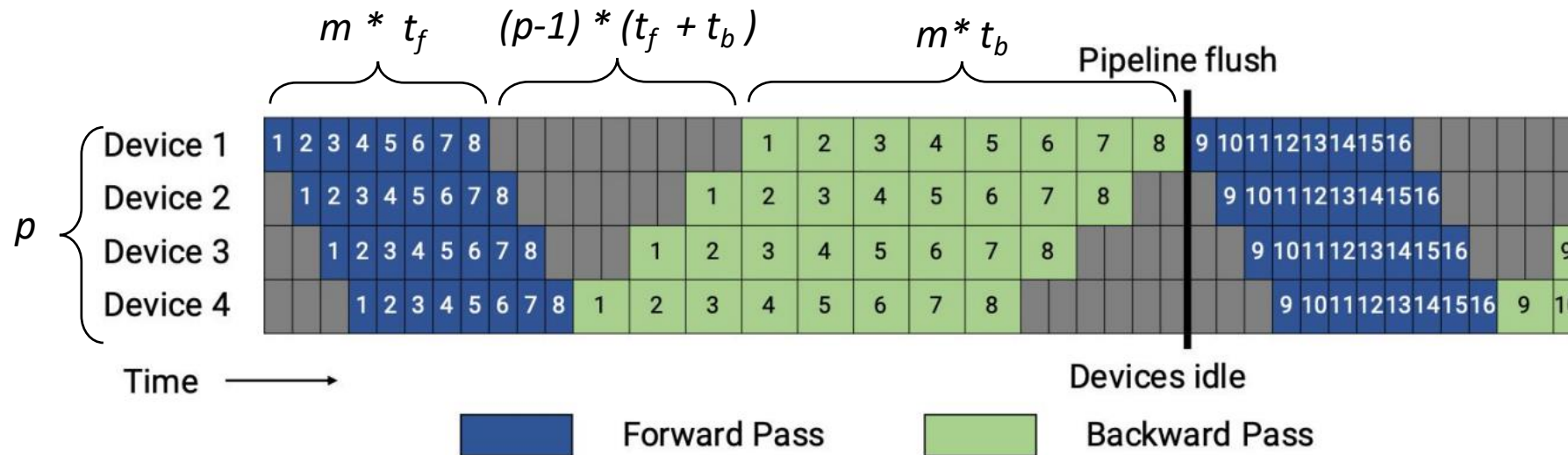


m = #microbatches (8)

p = pipeline stages (4)

t_f = time of forward

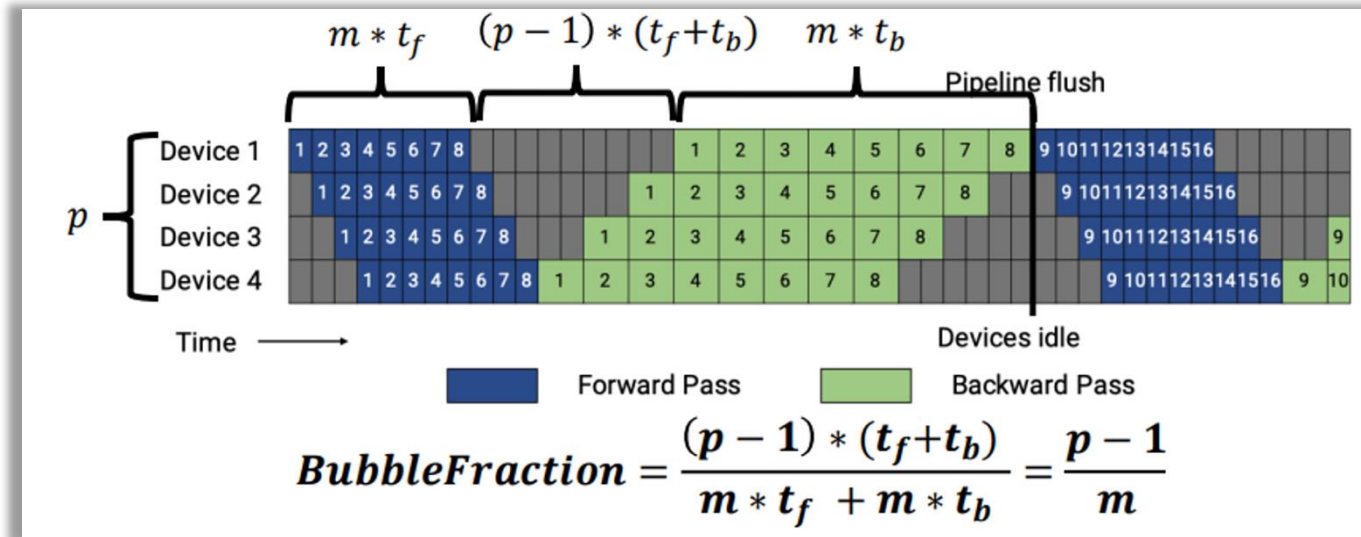
t_b = time of backward



$$\text{BubbleFraction} = \frac{(p-1) * (t_f + t_b)}{m * t_f + m * t_b} = \frac{p-1}{m}$$

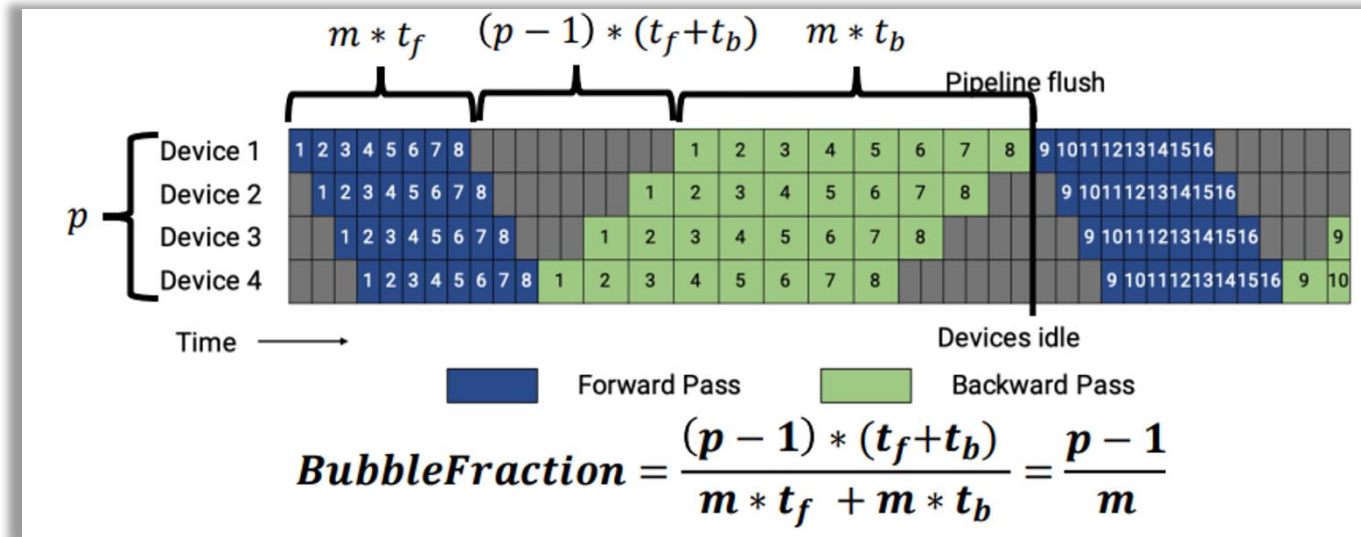
Question: How do we reduce the bubble fraction?

Improving Pipeline Parallelism Efficiency



- m : number of micro-batches in a mini-batch
 - Increase mini-batch size or reduce micro-batch size
 - Caveat: Increasing m reduces the micro-batch size, leaving insufficient work to saturate GPUs; moreover, m is capped by the mini-batch size

Improving Pipeline Parallelism Efficiency



- m : number of micro-batches in a mini-batch
 - Increase mini-batch size or reduce micro-batch size
 - Caveat: Increasing m reduces the micro-batch size, leaving insufficient work to saturate GPUs; moreover, m is capped by the mini-batch size
- p : number of pipeline stages
 - Decrease pipeline depth
 - Caveat: increase stage size

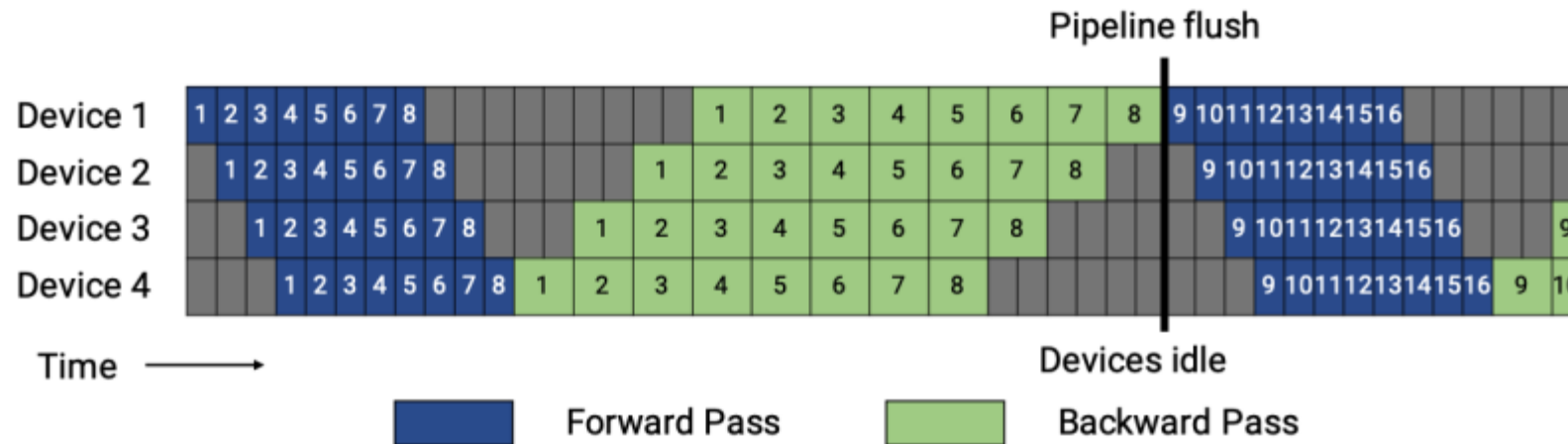
Question: How to further optimize pipeline parallelism?

- Each machine makes a choice between **two options**:
 - Perform the forward pass for a micro-batch, pushing the micro-batch to downstream workers
 - Perform the backward pass for a different micro-batch, ensuring forward progress in learning

Improving Memory Efficiency for Pipeline Parallelism



- An issue: we need to keep the intermediate activations of **all micro-batches** before back propagation

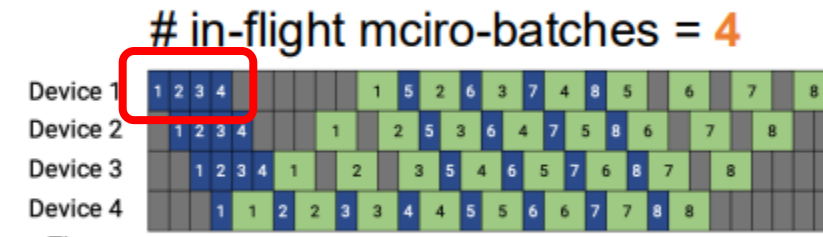


Question: Can we improve the pipeline schedule to reduce memory requirements?

- One-Forward-One-Backward in the steady state



Pipeline parallelism with GPipe's schedule



Pipeline parallelism with 1F1B schedule

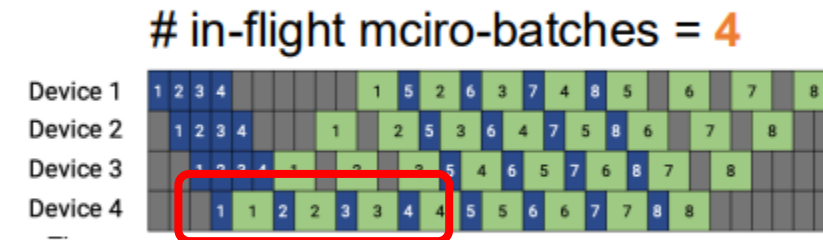
Pipeline Parallelism with 1F1B Schedule



- One-Forward-One-Backward in the steady state



Pipeline parallelism with GPipe's schedule



Pipeline parallelism with 1F1B schedule

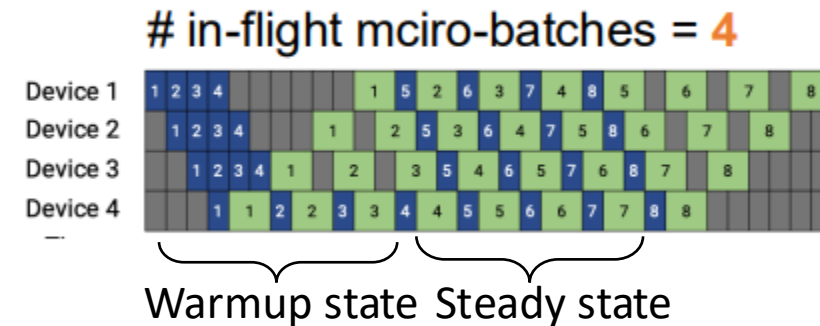
Pipeline Parallelism with 1F1B Schedule



- One-Forward-One-Backward in the steady state



Pipeline parallelism with GPipe's schedule



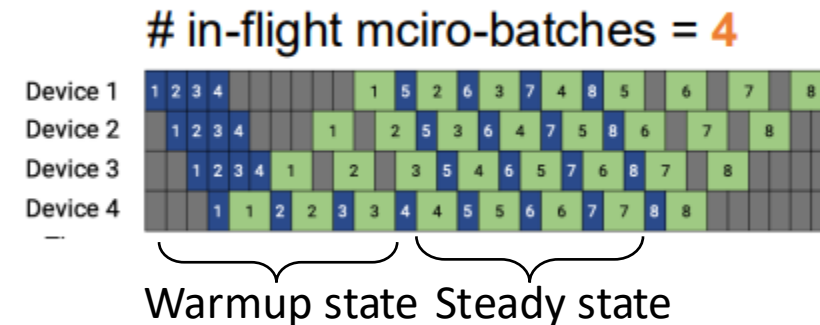
Pipeline parallelism with 1F1B schedule

- One-Forward-One-Backward in the steady state
- Reduce memory footprint of pipeline parallelism
- Doesn't reduce pipeline bubble

Can we reduce pipeline bubble?



Pipeline parallelism with GPipe's schedule



Pipeline parallelism with 1F1B schedule

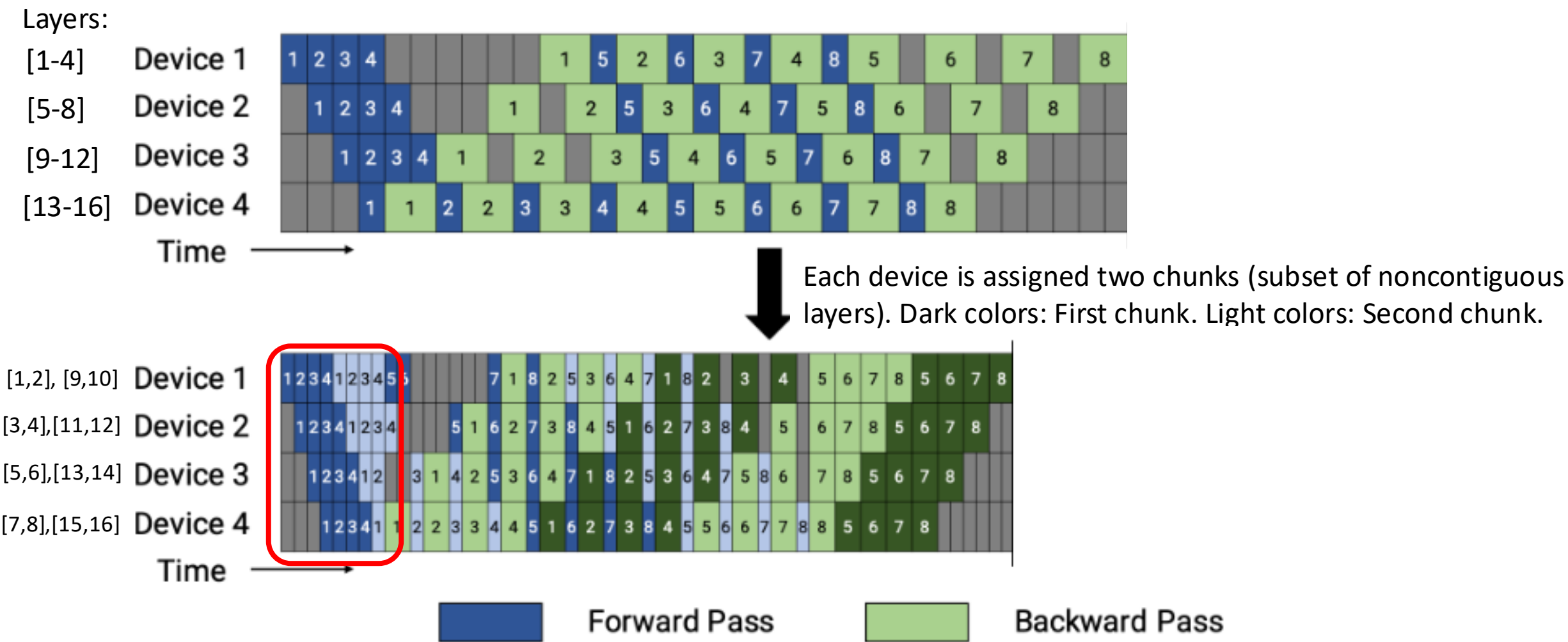
Pipeline Parallelism with Interleaved 1F1B Schedule



Layers:



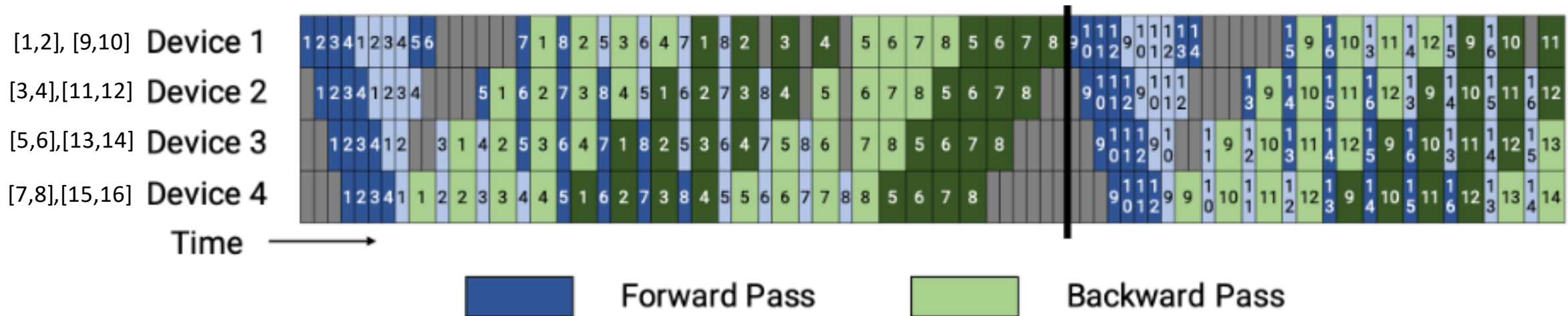
Pipeline Parallelism with Interleaved 1F1B Schedule



Pipeline Parallelism with Interleaved 1F1B Schedule



- Further divide each stages into v sub-stages
- The forward (backward) time of each sub-stage is t/v



Pipeline Parallelism with Interleaved 1F1B Schedule



- Further divide each stages into v sub-stages
- The forward (backward) time of each sub-stage is t/v

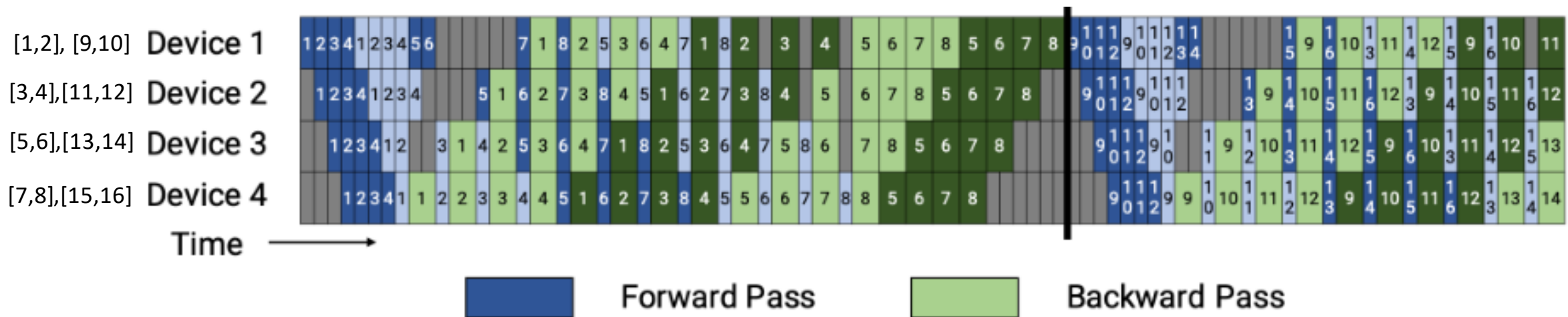
m = #microbatches

p = pipeline stages

t_f = time of forward

t_b = time of backward

$$\text{BubbleFraction} = \frac{(p-1) * \frac{(t_f+t_b)}{v}}{m * t_f + m * t_b} = \frac{1}{v} * \frac{p-1}{m}$$



Pipeline Parallelism with Interleaved 1F1B Schedule



- Further divide each stages into v sub-stages
- The forward (backward) time of each sub-stage is t/v

m = #microbatches

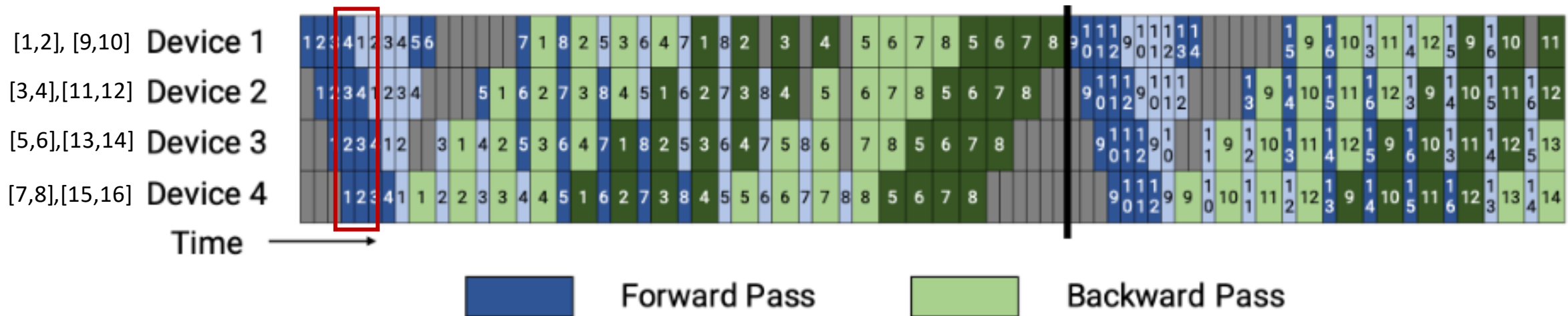
p = pipeline stages

t_f = time of forward

t_b = time of backward

$$\text{BubbleFraction} = \frac{(p-1) * \frac{(t_f+t_b)}{v}}{m * t_f + m * t_b} = \frac{1}{v} * \frac{p-1}{m}$$

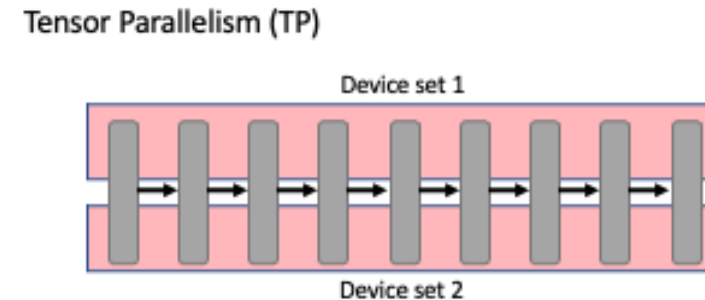
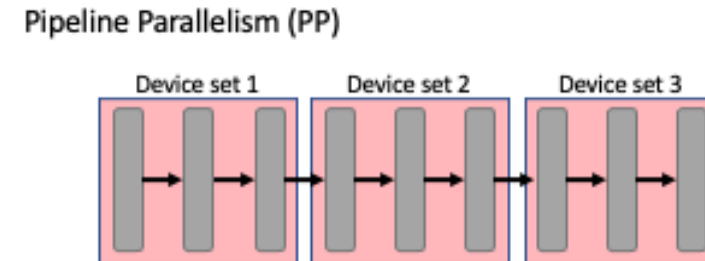
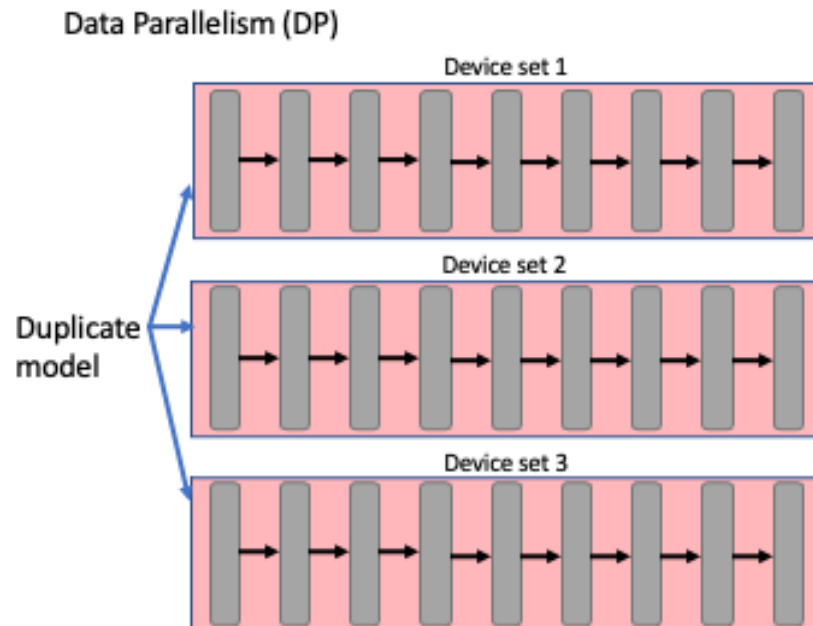
Reduce bubble time (increasing v) at the cost of increased communication



Summary: Pipeline Parallelism



- Data parallelism (DP) can not host a large model within a worker
- Tensor parallelism (TP) is often communication-intensive
- Pipeline parallelism (PP) introduces execution bubbles



Summary: Pipeline Parallelism



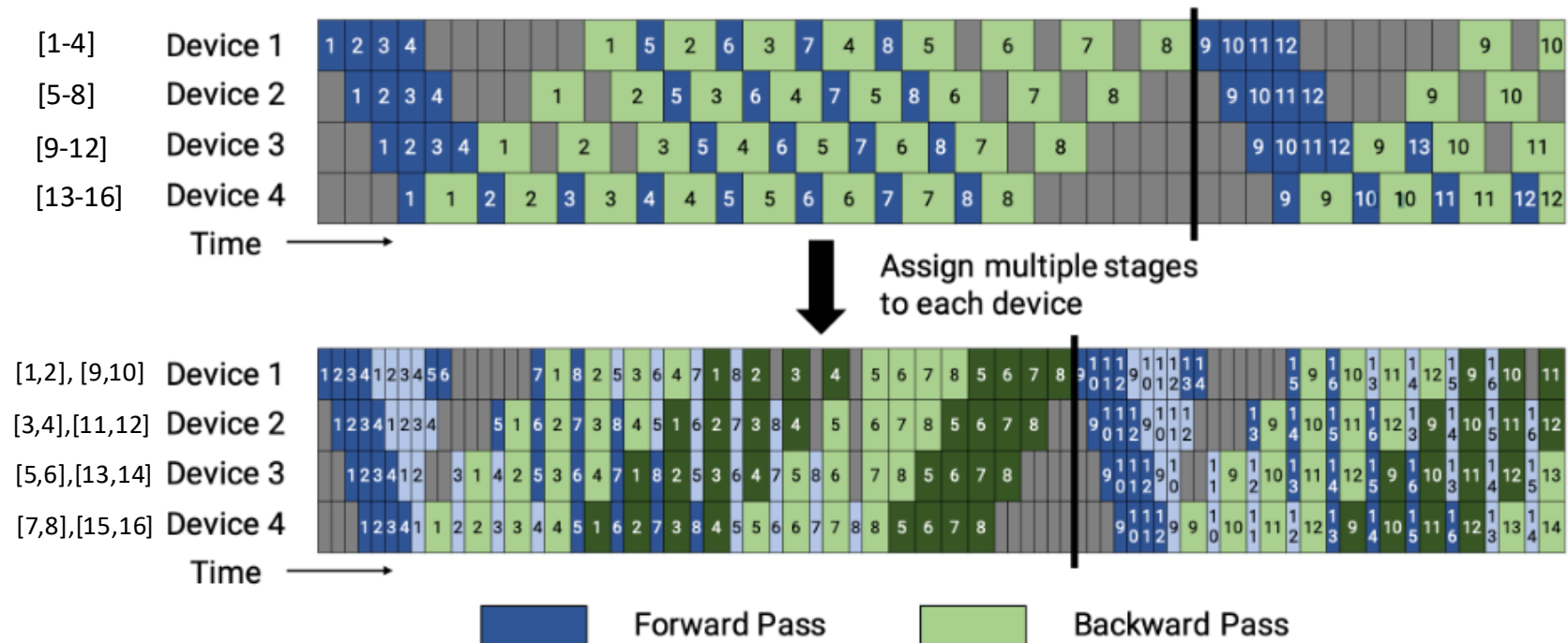
Pipeline parallelism with 1F1B Schedule

$$\text{BubbleFraction} = \frac{p-1}{m}$$

Pipeline parallelism with interleaved 1F1B Schedule

$$\text{BubbleFraction} = \frac{1}{v} * \frac{p-1}{m}$$

Layers:



Summary: Pipeline Parallelism



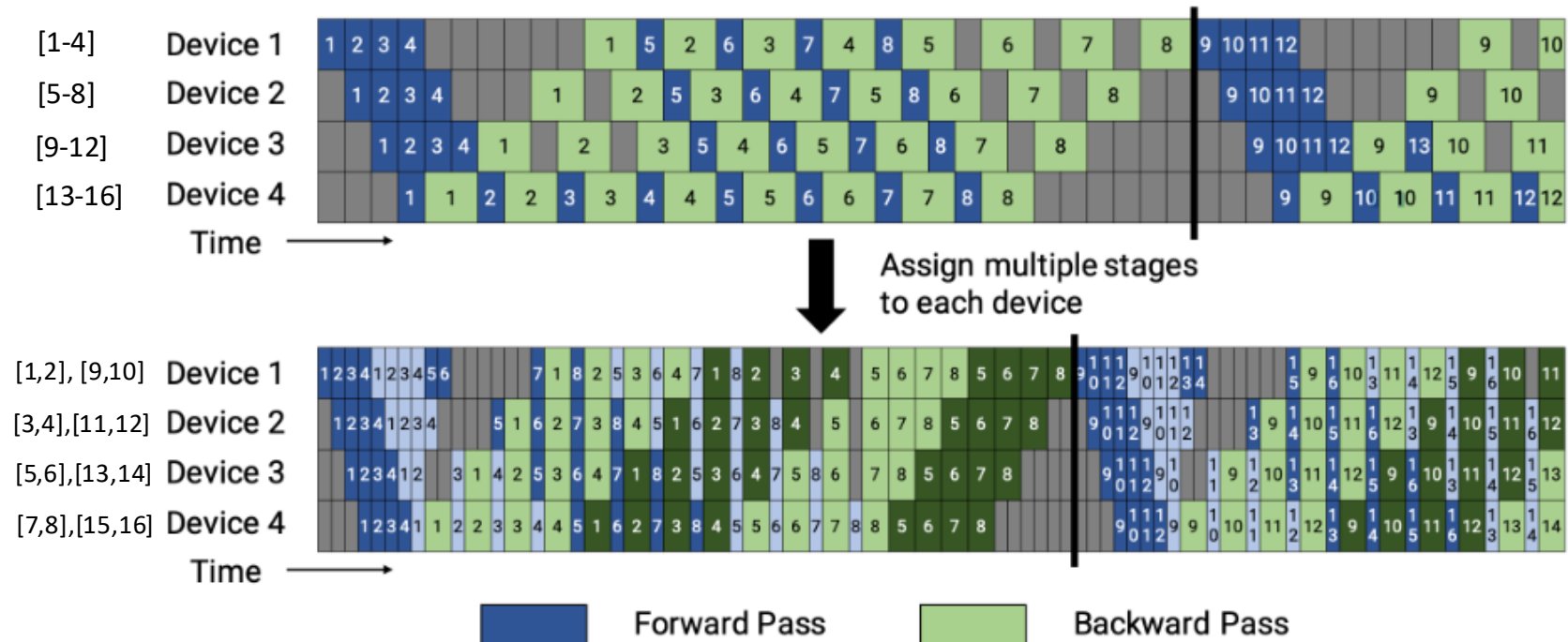
Pipeline parallelism with 1F1B Schedule

$$\text{BubbleFraction} = \frac{p-1}{m}$$

Pipeline parallelism with interleaved 1F1B Schedule

$$\text{BubbleFraction} = \frac{1}{v} * \frac{p-1}{m}$$

Layers:



LlamaRL: A Distributed Asynchronous Reinforcement Learning Framework for Efficient Large-scale LLM Training

Meta GenAI¹

LlamaRL: a streamlined, single-controller architecture built entirely on native PyTorch, enabling modularity, ease of use, and seamless scalability to thousands of GPUs.

Questions?